Growlithe — Time 26

Missões que fazem você voltar ao shopping. · MVP no ar: **growlithe-flax.vercel.app** · Código: **github.com/RohtiVM/26-growlithe**

1. Nome do projeto. Growlithe: web app gamificado de engajamento e fidelização para o Shopping Capim Dourado (Palmas, TO).

2. Problema identificado. A visita ao shopping é transacional: a pessoa compra o que precisava e vai embora, sem motivo para voltar. O shopping não sabe quem veio, com que frequência, nem quais ações funcionam. Programas de fidelidade tradicionais dão trabalho (cadastrar nota, cartão de pontos), e apps para instalar têm baixo alcance. Lojas de alas menos movimentadas não recebem fluxo.

3. Público-alvo. Visitantes do shopping (quem passeia, faz compras ou vai ao cinema); lojistas que querem mais gente na porta; a equipe de marketing do shopping, que precisa de dados de circulação e retorno.

4. Solução proposta. Transformar a visita em um jogo: o visitante aponta a câmera do celular para QR Codes espalhados pelo shopping, ganha pontos, sobe de nível (Bronze, Prata, Ouro, Diamante) e completa missões que levam a lojas e eventos. Abre direto no navegador, **sem instalar nada**.

5. Descrição do produto (MVP).

- **Visitante:** escaneia o QR e recebe pontos na hora (check-in de entrada vale mais); entra só com o e-mail (link mágico, sem senha); vê nível, saldo, presença no shopping e missões (caça ao tesouro em ordem, check-in em evento, Missão Relâmpago com pontos multiplicados).
- **Painel do shopping:** cadastra pontos físicos e imprime os QR Codes; cria e encerra missões; lojista valida o cupom de resgate de 6 dígitos; dashboard com check-ins, visitantes, scans por checkpoint e resultado das ações.
- **Landing page (prática sorteada):** apresenta o produto aos três públicos, teste A/B de manchete e lista de espera que mede a conversão de cada variante.
- **Easter egg de sexta-feira 13:** de madrugada o mascote aparece dormindo; com chuva forte em Palmas, gotas escorrem pela tela e o app destaca lojas de guarda-chuva e bebidas quentes.

6. Tecnologias utilizadas. TypeScript (strict) · React 19 com Next.js 15 · Supabase (PostgreSQL, autenticação e regras de acesso por linha) · Zod para validar toda entrada · Vercel (hospedagem, funções em São Paulo) · GitHub Actions (tipos, lint, 188 testes, testes de banco e build em todo PR) · Open-Meteo (clima) · desenvolvimento orientado a especificação (spec → tickets → testes primeiro → revisão), com 6 specs e 18 pull requests aprovados pelo CI.

7. Diferencial e inovação. Zero instalação: a câmera nativa do celular já abre o app. **QR Code à prova de fraude:** cada código é assinado e conferido dentro do banco; um mesmo ponto só pontua uma vez por dia, mesmo com requisições simultâneas. Recompensa **comportamento** (circular, voltar, ir a eventos), não só gasto. Para o shopping, cada scan vira dado de circulação que ele hoje não tem. Privacidade desde o início (LGPD): o lojista só vê o primeiro nome e o nível do visitante.

8. Viabilidade e aplicação no shopping. Custo de implantação quase zero: os QR Codes são impressos pelo próprio painel e colados em lojas, entradas e palcos; a infraestrutura roda no plano gratuito. Não exige integração com os sistemas dos lojistas. O marketing cria uma Missão Relâmpago em minutos para movimentar uma ala vazia e acompanha o resultado no dashboard. **Próximo passo:** validar compras pelo QR do cupom fiscal (NFC-e), que toda loja já imprime — já especificado e modelado (spec 001).